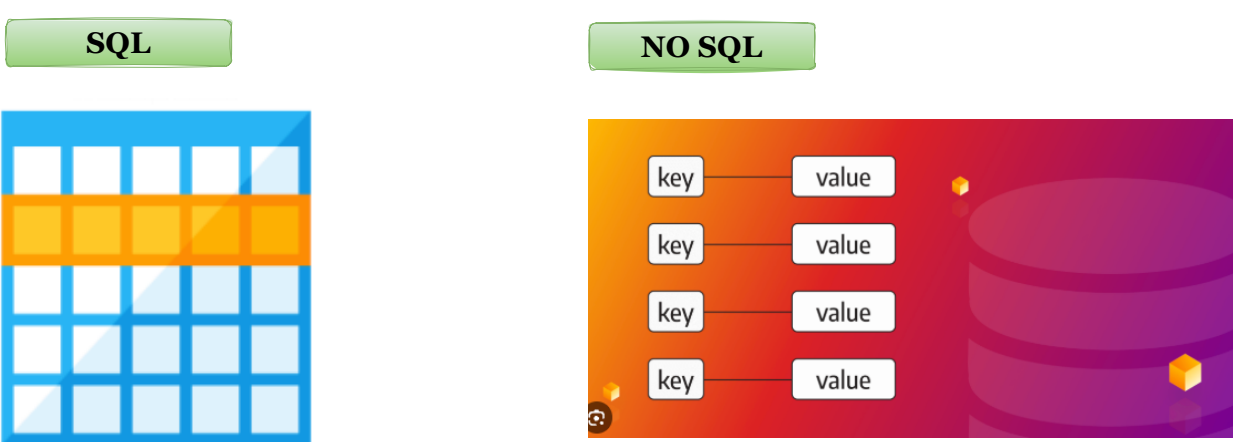
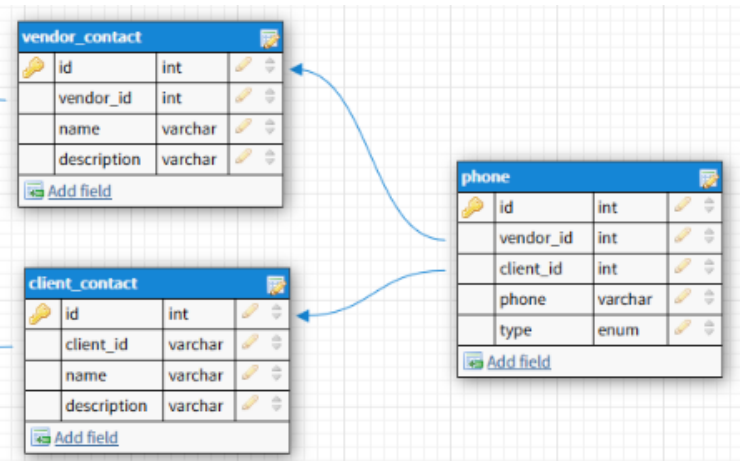


why do we need graph db ?



Problem

As data grows - Complexity grows

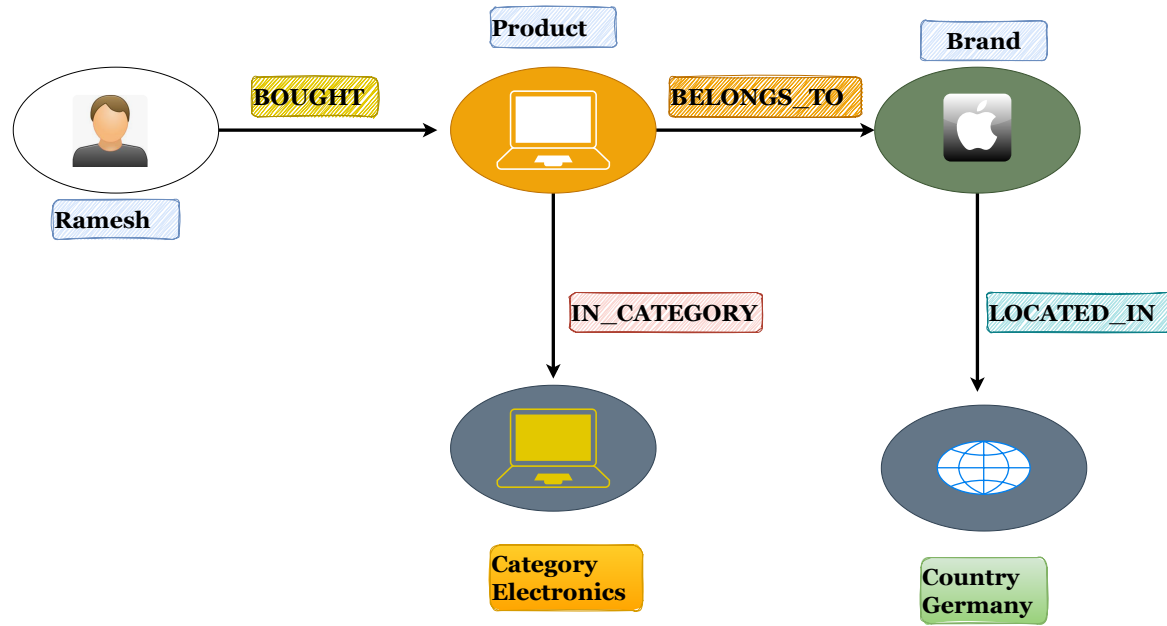


Now Imagine if I want to define the relation between these tables to find the client relationship ?? it become so hard and I have to perform diff db operations like Joins, multiple queries, recursive queries and so on ..

Solution

This is where we have our graph db which is handy to define the relation ships

GRAPH - DB



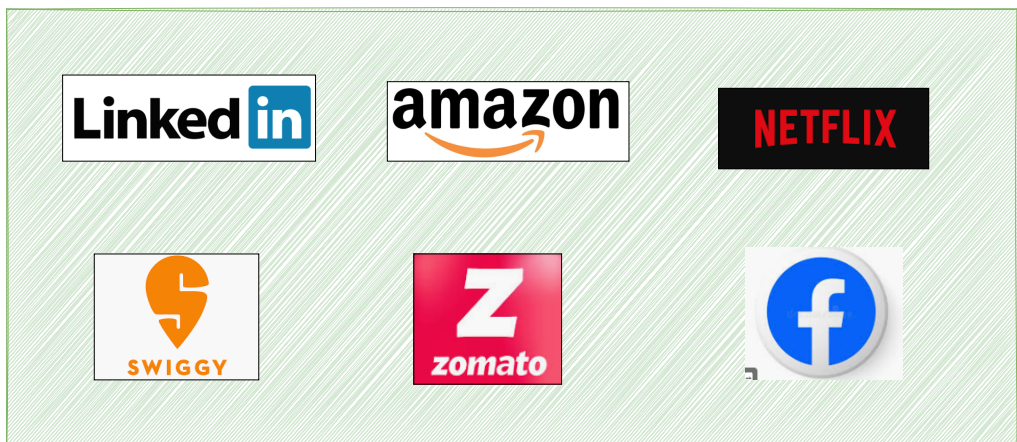
Examples

Query → Find users who bought products from brands located in Germany

In SQL: Multiple joins, more complex queries

In GraphDB: Natural traversal across connected nodes

Real Examples



1. Facebook GraphDB Example

Facebook is all about relationships:

People
Friends
Posts
Pages
Groups
Likes
Comments
Shares

Graph Model

(User)-[:FRIEND_OF]->(User)
(User)-[:POSTED]->(Post)
(User)-[:LIKED]->(Post)
(User)-[:MEMBER_OF]->(Group)
(User)-[:FOLLOWS]->(Page)

Example Graph

Ramesh-[:FRIEND_OF]-> Suresh
ramesh-[:LIKED]-> Post_101
Suresh-[:COMMENTED_ON]-> Post_101
Ramesh-[:MEMBER_OF]-> GenAI_Group

If we want to recommend friends

Who are friends of my friends, but not already my friends?

Neo4j - Cypher

```
MATCH (me:User {name: "Ramesh"})-[:FRIEND_OF]->(friend)-[:FRIEND_OF]->(suggested)
WHERE NOT (me)-[:FRIEND_OF]->(suggested)
AND me <> suggested
RETURN suggested.name AS suggested_friend, count(*) AS mutual_friends
ORDER BY mutual_friends DESC;
```

Explanation

Facebook can use graph relationships to find mutual friends.
If 10 of my friends know the same person, Facebook may suggest that person as a friend.

2. LinkedIn GraphDB Example

LinkedIn is a professional graph:

People
Companies
Skills
Jobs
Schools
Connections
Recommendations

Graph Model

(Person)-[:CONNECTED_TO]->(Person)
(Person)-[:WORKS_AT]->(Company)
(Person)-[:HAS_SKILL]->(Skill)
(Person)-[:APPLIED_TO]->(Job)
(Job)-[:REQUIRES_SKILL]->(Skill)
(Company)-[:POSTED]->(Job)

Example Graph

Shaan-[:HAS_SKILL]-> LangChain
Shaan-[:HAS_SKILL]-> GenAI
Job_123-[:REQUIRES_SKILL]-> LangChain
Job_123-[:REQUIRES_SKILL]-> RAG
Job_123-[:POSTED_BY]-> OpenAI_Startup

Question:

Which jobs match my skills?

Neo4j - Cypher

```
MATCH (p:Person {name: "Shaan"})-[:HAS_SKILL]->(skill)-[:REQUIRES_SKILL]->(job:Job)
RETURN job.title AS job_title, count(skill) AS matched_skills
ORDER BY matched_skills DESC;
```

Explanation

LinkedIn can recommend jobs by matching your skills with job-required skills.
This is not just keyword search.
It is relationship matching.

3. Netflix GraphDB Example

Netflix can model relationships between:

Users
Movies
Actors
Directors
Genres
Watch history
Ratings
Similar content

Graph Model

(User)-[:WATCHED]->(Movie)
(User)-[:RATED]->(Movie)
(Movie)-[:HAS_GENRE]->(Genre)
(Movie)-[:ACTED_IN]->(Actor)
(Movie)-[:DIRECTED_BY]->(Director)
(Movie)-[:SIMILAR_TO]->(Movie)

Example Graph

Rajesh-[:WATCHED]-> Money_Heist
Money_Heist-[:HAS_GENRE]-> Crime
Money_Heist-[:HAS_GENRE]-> Thriller
Money_Heist-[:SIMILAR_TO]-> Lupin
Lupin-[:HAS_GENRE]-> Heist

Question:

Recommend movies similar to what I watched.

Neo4j - Cypher

```
MATCH (u:User {name: "Rajesh"})-[:WATCHED]->(m:Movie)-[:HAS_GENRE]->(g:Genre)-[:HAS_GENRE]->(rec:Movie)
WHERE NOT (u)-[:WATCHED]->(rec)
RETURN rec.title AS recommended_movie, count(g) AS matching_genres
ORDER BY matching_genres DESC;
```

Explanation

Netflix can recommend movies based on shared genres, actors, directors, and viewing behavior.
GraphDB helps connect user taste with content features.